Contours & Shape Analysis

Week 9 — Session 2: Descriptors, invariance & hierarchy

Practical Image Processing

Lecture 9.2 ▪ 60–70 min

Recap of Session 1

- We traced **contours** from a binary mask and measured them: area (shoelace), perimeter, bounding box, rotated box, centroid.
- We filtered noise by **area** + **aspect ratio**, then cropped and ordered digits.
- Today: numbers that *describe* a shape and stay stable under rotation and scale — plus convex hulls, polygon approximation, and hierarchy (holes).

DocScan milestone

Approximate a page's outline to *4 corners* — the input to Week 11's perspective flatten.

Roadmap

Convex hull & convexity

Shape descriptors

Hu moments & matching

Approximation & hierarchy

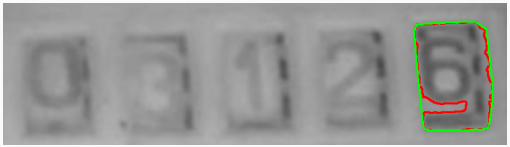
Deep dive

In-class exercise

Wrap-up & deliverable

Convex hull & convexity

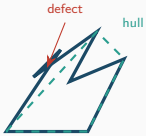
Convex hull & convexity



contour (red) + convex hull (green)

- **Convex hull**: the smallest convex shape enclosing the contour — a rubber band snapped around it.
- `hull = cv2.convexHull(c)` returns the hull points.
- **Convexity defects** (dents) reveal fingers, gaps, or the neck between touching objects.
- **Solidity** = area / hull area — how "filled" the shape is.

Convexity defects



- A **defect** is a stretch where the contour dips away from its hull; its *depth* measures how deep the dent is.
- `cv2.convexityDefects` returns start / end / farthest point / depth per dent.
- Classic use: count fingers on a hand, or detect where two coins touch (a deep neck).

For DocScan

A clean page has *few, shallow* defects — a strong "this really is a rectangle" cue.

Worked example: solidity

Solidity

$$\text{solidity} = \frac{\text{contour area}}{\text{hull area}}$$

A solid convex blob $\rightarrow$ 1.0; a dented / spiky shape $\rightarrow$ well below 1.

- Star shape: area = 120, hull area = 200 $\Rightarrow$ solidity = 0.60.
- Filled disc: area $\approx$ hull area $\Rightarrow$ solidity $\approx$ 0.98.
- A digit "8" has a lowish solidity (two holes eat area); a filled "1" is high.

Good practice

Solidity is a cheap, scale-free feature — great for separating spiky junk from compact digits.

Shape descriptors

Descriptors: numbers that describe a shape

- **Circularity** $= 4\pi\,\mathrm{area}/\mathrm{perimeter}^2$ (1 for a circle, lower for complex shapes).
- **Aspect ratio** $= w/h$, **extent** $=$ area/box area, **solidity** $=$ area/hull area.
- **Moments**: weighted sums that summarize a shape (centroid, orientation, spread).

Key idea

A descriptor maps a whole shape to one small vector of numbers you can compare and threshold.

Circularity — derivation & worked values

Why $4\pi A/P^2$?

A circle has $A = \pi r^2$, $P = 2\pi r$, so

$$\frac{4\pi A}{P^2} = \frac{4\pi(\pi r^2)}{(2\pi r)^2} = \frac{4\pi^2 r^2}{4\pi^2 r^2} = 1.$$

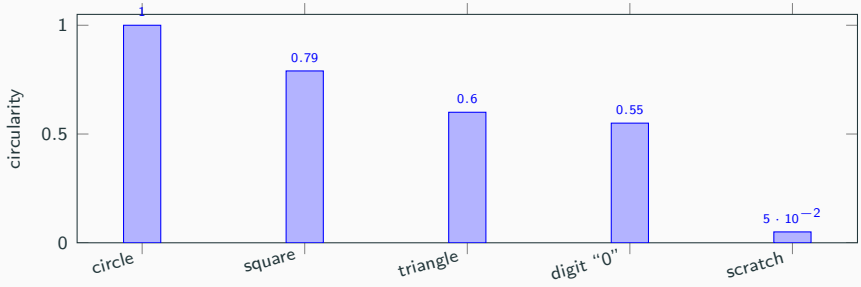
Any other shape has more perimeter per unit area $\Rightarrow$ value < 1.

- Square: $A = s^2$, $P = 4s \Rightarrow 4\pi s^2/16s^2 = \pi/4 \approx \mathbf{0.79}$.
- Equilateral triangle: $\approx \mathbf{0.60}$.
- Long thin scratch: P huge, A tiny $\Rightarrow \approx \mathbf{0.05}$.

Also called

"roundness" or (its inverse) the **isoperimetric** ratio.

Circularity separates shapes



- One threshold (~ 0.2) already rejects scratches; combine with area for robust filtering.

Extent, aspect ratio, solidity — side by side

Descriptor	Formula	Range	Reads as
Aspect ratio	w/h	$0 \to \infty$	tall vs wide
Extent	area $/(w \cdot h)$	$0 \to 1$	fills its box?
Solidity	area / hull area	$0 \to 1$	dented / spiky?
Circularity	$4\pi A/P^2$	$0 \to 1$	round vs jagged

✔ Good practice

All four are *ratios* — unit-free and scale-free. A shape twice as big gives the same values.

Moments: the raw material of descriptors

Image moments

$$M_{pq} = \sum_x \sum_y x^p \, y^q \, I(x, y)$$

M_{00} = area/mass; M_{10}, M_{01} give the centroid $(\bar{x}, \bar{y}) = (M_{10}/M_{00},\ M_{01}/M_{00})$.

- **Central moments** $\mu_{pq} = \sum (x - \bar{x})^p (y - \bar{y})^q I$ subtract the centroid $\Rightarrow$ **translation-invariant**.
- **Normalized** central moments $\eta_{pq} = \mu_{pq} / \mu_{00}^{(1+(p+q)/2)}$ add **scale** invariance.

Orientation from second moments

Principal axis

$$\theta = \tfrac{1}{2} \arctan\left(\frac{2\mu_{11}}{\mu_{20} - \mu_{02}}\right)$$

the angle of the shape's long axis — its "tilt."



- Second central moments $\mu_{20}, \mu_{02}, \mu_{11}$ describe how the mass *spreads*.
- Orientation lets you de-rotate a tilted digit before recognition.

Hu moments & matching

Hu moments: rotation/scale-invariant shape

- **Hu moments**: 7 numbers built from normalized central moments, (nearly) invariant to **translation, rotation, and scale**.
- So the same shape gives the same descriptor regardless of where, how big, or how rotated it is.
- A classical alternative to a neural net for matching a shape to a template.

First Hu moment

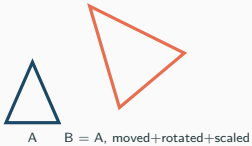
$h_1 = \eta_{20} + \eta_{02}$ — spread about the centroid; the rest combine higher η_{pq} to cancel rotation.

matchShapes: a shape similarity score

```
hu = cv2.HuMoments(cv2.moments(c)).flatten()   # 7-vector
# compare two contours directly:
score = cv2.matchShapes(c1, c2, cv2.CONTOURS_MATCH_I1, 0.0)
# score ~ 0  => very similar;  larger => more different
```

- Hu values span many orders of magnitude $\Rightarrow$ compare with a **log** transform (matchShapes does this internally).
- Match a query digit against templates "0"–"9"; smallest score wins — a tiny, training-free classifier.

Worked example: same shape, moved & rotated



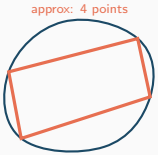
- Raw moments of A and B differ wildly (different position, size, angle).
- **Hu moments** of A and B are (almost) identical $\Rightarrow$ `matchShapes` score ≈ 0.
- Small residual comes from rasterization, not from the transform.

Key idea

Invariance means the descriptor answers "what shape?" not "where / how big / which angle?"

Approximation & hierarchy

Contour approximation (polygon fitting)



- `approxPolyDP(c, eps, True)` simplifies a contour to fewer points within a tolerance `eps`.
- Pick `eps` as a fraction of perimeter: `eps = 0.02*cv2.arcLength(c,True)`.
- A page/document boundary → approximates to *4 points* = the corners.

Key idea

Approximating the page contour to a quadrilateral is how DocScan finds the page's corners.

Douglas–Peucker: how the simplification works



1. Draw a line from the first to the last point; find the point farthest from it.
2. If that distance $>$ `eps`, keep the point and recurse on the two halves.
3. Otherwise discard all points between — a straight segment is enough.

Tuning

Bigger `eps` $\Rightarrow$ fewer points (coarser). Too big and a rectangle loses a corner.

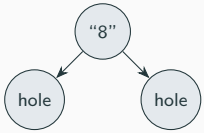
DocScan: page → 4 corners

- Take the *largest* external contour (the page), approximate it, and *require* 4 points.
- If the approximation gives 4 points → you have the page corners; order them TL, TR, BR, BL.
- Those 4 corners feed the perspective correction in Week 11 — the DocScan "flatten" step.

✕ Pitfall

Shadows or a busy background can make the "page" contour non-quadrilateral. Guard: check area, convexity, and that exactly 4 points came back before trusting it.

Contour hierarchy (holes)



outer contour + 2 inner (holes)

- **RETR_TREE** returns a `hierarchy` array: `[next, prev, first_child, parent]` per contour.
- A “0” has one hole; “8” has two; “1” has none — a cheap, discriminative feature.
- For plain digit cropping, **RETR_EXTERNAL** (outer only) is usually enough.

Counting holes as a feature

Digit	1	4	6	8	0
# holes	0	0/1	1	2	1

- Count children of a contour in the hierarchy $\Rightarrow$ number of enclosed holes.
- Combined with solidity and aspect ratio, holes alone already split many digit classes.
- Font-dependent (a closed "4") — a hint, not a full classifier.

Deep dive

Counting & measuring real objects

- Count = number of valid contours after filtering — cells, coins, parts, digits.
- Measure = area / length / diameter per contour, converted to real units via a known scale.
- This is the backbone of industrial inspection and microscopy — pure classical vision.

✔ Good practice

A calibrated pixel-to-mm ratio turns contour measurements into real-world dimensions.

Worked example: pixel → millimetre

Calibration

A ruler's 10 mm reads as 200 px $\Rightarrow$ scale $s = \dfrac{10}{200} = 0.05$ mm/px.

- A part's contour: width = 120 px $\Rightarrow$ $120 \times 0.05 = \mathbf{6.0}$ mm.
- Area scales by s^2: $4000\,\text{px}^2 \times 0.05^2 = \mathbf{10}\,\text{mm}^2$.

✗ Pitfall

Area uses s^2, not s. Forgetting the square is the classic calibration bug.

Robustness: the real-world contour mess

- Contours are only as good as the binary mask — garbage in, garbage out.
- Broken digit $\rightarrow$ two contours; merged digits $\rightarrow$ one. Fix upstream (Weeks 7–8).
- Always filter by shape *and* sanity-check the count on real data.

✕ Pitfall

Don't trust the contour count blindly — a shadow or a smudge can add or merge objects.

Contours in code — the full loop

```python
cnts, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
digits = []
for c in cnts:
    x, y, w, h = cv2.boundingRect(c)
    if cv2.contourArea(c) > 200 and 0.2 < w/h < 1.2:   # keep digit-like
        digits.append((x, gray[y:y+h, x:x+w]))
digits = [img for x, img in sorted(digits)]            # left-to-right
```

Segment → contour → filter → crop → order. The DocScan hand-off, in seven lines.

Fourier descriptors (brief)

- Represent a contour as a complex signal $z_k = x_k + iy_k$ and take its Fourier transform (Week 6 again).
- Low-frequency coefficients = overall shape; high = fine boundary detail.
- Normalizing the coefficients gives **translation / rotation / scale** invariance, like Hu moments.
- Keeping a few coefficients gives a compact, robust descriptor for matching.

Shape analysis → recognition

- Hu moments / Fourier descriptors can classify simple shapes with *no* training.
- For digits, hand-crafted descriptors work but a CNN (Week 13) is more accurate.
- The pipeline is the same: segment → contour/crop → describe/classify.

Key idea

Contours give clean, isolated objects; classical descriptors or a CNN then name them.

In-class exercise

In-class exercise (12 min)

Pick descriptors for a task

You must sort a tray of mixed hardware from one photo: round washers, hex nuts, and long screws.

1. Which *single* descriptor best separates washers from screws? Why?
2. Which distinguishes a washer (has a hole) from a solid disc?
3. Two nuts overlap and merge into one contour — which cue (defects? solidity?) flags it?
4. Which of your chosen descriptors survive the camera zooming in $2\times$?

Debrief

- Washer vs screw: **circularity** (or aspect ratio) — screws are long and thin, washers round.
- Washer vs disc: hole count from the **hierarchy**, or low **solidity** / extent.
- Merged nuts: a deep **convexity defect** at the neck; solidity drops below a single nut's.
- Zoom-safe: all *ratio* descriptors (circularity, solidity, extent, Hu) are scale-free. ✓

Key idea

Choose descriptors by the invariances the task needs — not by habit.

Wrap-up & deliverable

What you should now be able to do

- Compute convex hull, convexity defects, and solidity.
- Compute shape descriptors (circularity, extent, solidity, aspect ratio) and explain their invariances.
- Use moments for centroid & orientation; use Hu moments / `matchShapes` for invariant matching.
- Approximate a page contour to 4 corners; read a contour hierarchy to count holes.

This week's deliverable

Due before Week 10 (see `project/`)

1. Find and draw contours on a cleaned meter binary; report each digit's area & bounding box.
2. Filter out noise contours with an area + aspect-ratio rule; verify the count.
3. Crop each digit by its bounding box and output them ordered left-to-right.
4. Compute one **invariant** descriptor per digit (circularity or a Hu moment) and tabulate it.
5. Bonus: approximate a document's outer contour to its 4 corners.

From outlines to identities.

Describe, make invariant, approximate, and match — the classical road to reading a shape.